`ASPNETCORE_FORWARDEDHEADERS_ENABLED=true`, as you saw in chapter 27. Alternatively, you can add the middleware to your application manually, as shown in section 27.3.2.

When the reverse proxy forwards a request, the `ForwardedHeadersMiddleware` looks for the `X-Forwarded-Proto` header and updates the request details as appropriate. For all subsequent middleware, the request is considered secure. When adding the middleware manually, it's important that you place `ForwardedHeadersMiddleware` before the call to `UseHsts()` or `UseHttpsRedirection()` so that the forwarded headers are read and the request is marked secure, as appropriate.

Using the HSTS and HTTPS redirection middleware is best practice when you're building a server-side application such as a Razor Pages app that will always be accessed in the browser. If you're building an API application. however, a better approach is to not listen for insecure HTTP requests at all!

28.4.3 Rejecting HTTP requests in API applications

Browsers have been adding more and more protections, such as the HSTS header, to try to protect users from using insecure HTTP requests. But not all clients are using a web browser. In this section you'll learn why API applications should generally disable HTTP entirely.

If you're building an API application, you often can't rely on requests coming from a browser. Your API application may primarily serve a client-side framework in the browser, but it may also serve mobile applications or provide an API to other backend services. That means you can't rely on the protections built into web browsers to use HTTPS for your API apps.

On top of that, even if you know all your users are using a browser, the only way to prevent sending all requests over HTTP is to use HSTS preload, as you saw in section 28.4.2. Sending even one request over HTTP can